Typing Rules, Validity, Injectivity, and Subject Reduction for Dependent Type Theory with Π, Σ, U:U, and Prop:U

1 Judgments

We work with three mutually defined judgment forms:

$\Gamma\ \mathsf{ctx}$	Γ is a well-formed context
$\Gamma \vdash M : A$	M has type A in context Γ
$\Gamma \vdash M = N : A$	M and N are definitionally equal at type A

Contexts are snoc-lists: $\Gamma ::= \cdot \mid \Gamma, x{:}A$. We write $M\sigma$ for the result of applying a substitution σ to a term M, and $[t]$ for the substitution $\Gamma \to \Gamma, x{:}A$ extending the identity by t, so that $B[a]$ means $B(\mathrm{id}, a)$.

2 Well-formed contexts

$$\frac{}{\cdot\ \mathsf{ctx}} \qquad\qquad \frac{\Gamma \vdash A : \mathsf{U}}{\Gamma, x{:}A\ \mathsf{ctx}}$$

3 Typing rules

Core rules

$$\textsc{Var}\ \frac{\Gamma\ \mathsf{ctx} \qquad (x{:}A) \in \Gamma}{\Gamma \vdash x : A} \qquad \textsc{Conv}\ \frac{\Gamma \vdash M : A \qquad \Gamma \vdash A = B : \mathsf{U} \qquad \Gamma \vdash B : \mathsf{U}}{\Gamma \vdash M : B} \qquad \textsc{U}\ \frac{\Gamma\ \mathsf{ctx}}{\Gamma \vdash \mathsf{U} : \mathsf{U}}$$

Pi types

$$\textsc{Pi}\ \frac{\Gamma \vdash A : \mathsf{U} \qquad \Gamma, x{:}A \vdash B : \mathsf{U}}{\Gamma \vdash \Pi(x{:}A)B : \mathsf{U}} \qquad \textsc{Lam}\ \frac{\Gamma \vdash A : \mathsf{U} \qquad \Gamma, x{:}A \vdash B : \mathsf{U} \qquad \Gamma, x{:}A \vdash M : B}{\Gamma \vdash \lambda(x{:}A)M : \Pi(x{:}A)B}$$

$$\textsc{App}\ \frac{\Gamma \vdash A : \mathsf{U} \qquad \Gamma, x{:}A \vdash B : \mathsf{U} \qquad \Gamma \vdash f : \Pi(x{:}A)B \qquad \Gamma \vdash a : A}{\Gamma \vdash f\ a : B[a]}$$

Sigma types

$$\textsc{Sigma}\ \frac{\Gamma \vdash A : \mathsf{U} \qquad \Gamma, x{:}A \vdash B : \mathsf{U}}{\Gamma \vdash \Sigma(x{:}A)B : \mathsf{U}} \qquad \textsc{MkPair}\ \frac{\Gamma \vdash A : \mathsf{U} \qquad \Gamma, x{:}A \vdash B : \mathsf{U} \qquad \Gamma \vdash M : A \qquad \Gamma \vdash N : B[M]}{\Gamma \vdash \mathsf{pair}(M, N) : \Sigma(x{:}A)B}$$

$$\textsc{Fst}\ \frac{\Gamma \vdash A : \mathsf{U} \qquad \Gamma, x{:}A \vdash B : \mathsf{U} \qquad \Gamma \vdash M : \Sigma(x{:}A)B}{\Gamma \vdash \mathsf{fst}(M) : A}$$

$$\textsc{Snd}\ \frac{\Gamma \vdash A : \mathsf{U} \qquad \Gamma, x{:}A \vdash B : \mathsf{U} \qquad \Gamma \vdash M : \Sigma(x{:}A)B}{\Gamma \vdash \mathsf{snd}(M) : B[\mathsf{fst}(M)]}$$

Propositions

$$\textsc{Prop}\ \frac{\Gamma\ \mathsf{ctx}}{\Gamma \vdash \mathsf{Prop} : \mathsf{U}} \qquad \textsc{Prop-U}\ \frac{\Gamma \vdash A : \mathsf{Prop}}{\Gamma \vdash A : \mathsf{U}} \qquad \textsc{Pi-Prop}\ \frac{\Gamma \vdash A : \mathsf{U} \qquad \Gamma, x{:}A \vdash B : \mathsf{Prop}}{\Gamma \vdash \Pi(x{:}A)B : \mathsf{Prop}}$$

4 Conversion rules

General

$$\textsc{Refl}\ \frac{\Gamma \vdash M : A}{\Gamma \vdash M = M : A} \qquad \textsc{Sym}\ \frac{\Gamma \vdash M = N : A}{\Gamma \vdash N = M : A} \qquad \textsc{Trans}\ \frac{\Gamma \vdash M = N : A \qquad \Gamma \vdash N = P : A}{\Gamma \vdash M = P : A}$$

$$\textsc{Conv-Eq}\ \frac{\Gamma \vdash M = N : A \qquad \Gamma \vdash A = B : \mathsf{U} \qquad \Gamma \vdash B : \mathsf{U}}{\Gamma \vdash M = N : B}$$

Pi computation and congruence

$$\textsc{Beta}\quad \frac{\Gamma \vdash A : \mathsf{U} \qquad \Gamma, x{:}A \vdash B : \mathsf{U} \qquad \Gamma, x{:}A \vdash M : B \qquad \Gamma \vdash a : A}{\Gamma \vdash (\lambda(x{:}A)M)\ a = M[a] : B[a]}$$

$$\textsc{Pi-Cong}\quad \frac{\Gamma \vdash A = A' : \mathsf{U} \qquad \Gamma, x{:}A \vdash B = B' : \mathsf{U}}{\Gamma \vdash \Pi(x{:}A)B = \Pi(x{:}A')B' : \mathsf{U}}$$

$$\textsc{Funext}\quad \frac{\Gamma \vdash A : \mathsf{U} \qquad \Gamma, x{:}A \vdash B : \mathsf{U} \qquad \Gamma, x{:}A \vdash f^+\ x = g^+\ x : B \qquad \Gamma \vdash f : \Pi(x{:}A)B \qquad \Gamma \vdash g : \Pi(x{:}A)B}{\Gamma \vdash f = g : \Pi(x{:}A)B}$$

$$\textsc{App-Fun}\quad \frac{\Gamma \vdash A : \mathsf{U} \qquad \Gamma, x{:}A \vdash B : \mathsf{U} \qquad \Gamma \vdash f = f' : \Pi(x{:}A)B \qquad \Gamma \vdash a : A}{\Gamma \vdash f\ a = f'\ a : B[a]}$$

$$\textsc{App-Arg}\quad \frac{\Gamma \vdash A : \mathsf{U} \qquad \Gamma, x{:}A \vdash B : \mathsf{U} \qquad \Gamma \vdash f : \Pi(x{:}A)B \qquad \Gamma \vdash a = a' : A}{\Gamma \vdash f\ a = f\ a' : B[a]}$$

Sigma computation and congruence

$$\textsc{Sigma-Cong}\ \frac{\Gamma \vdash A = A' : \mathsf{U} \qquad \Gamma, x{:}A \vdash B = B' : \mathsf{U}}{\Gamma \vdash \Sigma(x{:}A)B = \Sigma(x{:}A')B' : \mathsf{U}}$$

$$\textsc{Beta-Fst}\ \frac{\Gamma \vdash A : \mathsf{U} \qquad \Gamma, x{:}A \vdash B : \mathsf{U} \qquad \Gamma \vdash M : A \qquad \Gamma \vdash N : B[M]}{\Gamma \vdash \mathsf{fst}(\mathsf{pair}(M, N)) = M : A}$$

$$\textsc{Beta-Snd}\ \frac{\Gamma \vdash A : \mathsf{U} \qquad \Gamma, x{:}A \vdash B : \mathsf{U} \qquad \Gamma \vdash M : A \qquad \Gamma \vdash N : B[M]}{\Gamma \vdash \mathsf{snd}(\mathsf{pair}(M, N)) = N : B[M]}$$

$$\textsc{Pair-Eta}\ \frac{\Gamma \vdash A : \mathsf{U} \qquad \Gamma, x{:}A \vdash B : \mathsf{U} \qquad \Gamma \vdash M : \Sigma(x{:}A)B}{\Gamma \vdash \mathsf{pair}(\mathsf{fst}(M), \mathsf{snd}(M)) = M : \Sigma(x{:}A)B}$$

$$\textsc{MkPair-Fst}\ \frac{\Gamma \vdash A : \mathsf{U} \qquad \Gamma, x{:}A \vdash B : \mathsf{U} \qquad \Gamma \vdash M = M' : A \qquad \Gamma \vdash N : B[M] \qquad \Gamma \vdash N' : B[M']}{\Gamma \vdash \mathsf{pair}(M, N) = \mathsf{pair}(M', N') : \Sigma(x{:}A)B}$$

$$\textsc{MkPair-Snd}\ \frac{\Gamma \vdash A : \mathsf{U} \qquad \Gamma, x{:}A \vdash B : \mathsf{U} \qquad \Gamma \vdash M : A \qquad \Gamma \vdash N = N' : B[M]}{\Gamma \vdash \mathsf{pair}(M, N) = \mathsf{pair}(M, N') : \Sigma(x{:}A)B}$$

$$\textsc{Fst-Cong}\ \frac{\Gamma \vdash A : \mathsf{U} \qquad \Gamma, x{:}A \vdash B : \mathsf{U} \qquad \Gamma \vdash M = M' : \Sigma(x{:}A)B}{\Gamma \vdash \mathsf{fst}(M) = \mathsf{fst}(M') : A}$$

$$\textsc{Snd-Cong}\ \frac{\Gamma \vdash A : \mathsf{U} \qquad \Gamma, x{:}A \vdash B : \mathsf{U} \qquad \Gamma \vdash M = M' : \Sigma(x{:}A)B}{\Gamma \vdash \mathsf{snd}(M) = \mathsf{snd}(M') : B[\mathsf{fst}(M)]}$$

Propositions

$$\textsc{Prop-Irr}\ \frac{\Gamma \vdash A : \mathsf{Prop} \qquad \Gamma \vdash M : A \qquad \Gamma \vdash N : A}{\Gamma \vdash M = N : A} \qquad \textsc{Prop-U-Eq}\ \frac{\Gamma \vdash M = N : \mathsf{Prop}}{\Gamma \vdash M = N : \mathsf{U}}$$

$$\textsc{Pi-Prop-Cong}\ \frac{\Gamma \vdash A = A' : \mathsf{U} \qquad \Gamma, x{:}A \vdash B = B' : \mathsf{Prop}}{\Gamma \vdash \Pi(x{:}A)B = \Pi(x{:}A')B' : \mathsf{Prop}}$$

5 Head reduction

We define single-step head reduction $M \rightsquigarrow_1 N$ as the compatible closure of β-reduction at the head, extended with projection reductions for Sigma types:

$$\textsc{HR-Beta}\ \frac{}{(\lambda(x{:}A)M)\ a \rightsquigarrow_1 M[a]} \qquad \textsc{HR-App}\ \frac{M \rightsquigarrow_1 M'}{M\ a \rightsquigarrow_1 M'\ a} \qquad \textsc{HR-Beta-Fst}\ \frac{}{\mathsf{fst}(\mathsf{pair}(M,N)) \rightsquigarrow_1 M} \qquad \textsc{HR-Beta-Snd}\ \frac{}{\mathsf{snd}(\mathsf{pair}(M,N)) \rightsquigarrow_1 N}$$

$$\textsc{HR-Fst}\ \frac{M \rightsquigarrow_1 M'}{\mathsf{fst}(M) \rightsquigarrow_1 \mathsf{fst}(M')} \qquad \textsc{HR-Snd}\ \frac{M \rightsquigarrow_1 M'}{\mathsf{snd}(M) \rightsquigarrow_1 \mathsf{snd}(M')}$$

We write $M \rightsquigarrow^* N$ for the reflexive-transitive closure of $\rightsquigarrow_1$. As in the Pi-only system, the validity relation uses a wrapper $\mathsf{Red}\ \Gamma\ M\ N\ A$ with phantom context and type parameters.

6 Domain model

We describe the domain-theoretic semantics following Coquand–Huber (2018). The model interprets types as *codes* (finite elements of a Scott domain) and terms as elements approximated by these codes.

6.1 Finite elements

The set Code of *codes* is defined inductively, together with a set $\mathsf{FunCode}$ of *finite functions* (finite lists of input-output pairs):

$$u, v, a ::= \bot \mid \mathsf{U} \mid \mathsf{Prop} \mid \lambda g \mid \mathsf{Pi}(a, f) \mid \mathsf{Sigma}(a, f) \mid (u, v)$$
$$g, f, h ::= [] \mid (u, v) :: g$$

Here $\bot$ is the bottom element (no information), U and Prop are the codes for the universe and the propositional universe, λg is a function value with finite graph g, $\mathsf{Pi}(a, f)$ is a Pi-type code with domain code a and codomain function f, $\mathsf{Sigma}(a, f)$ is a Sigma-type code, and (u, v) is a pair code.

6.2 Evaluation

The evaluation of a finite function f at input u computes the *supremum* of all outputs whose keys are below u in the information ordering:

$$\mathsf{Eval}(f, u) = \bigsqcup \{ v_i \mid (u_i, v_i) \in f,\ u_i \leq u \}$$

The binary supremum $u \sqcup v$ combines compatible elements: $\bot \sqcup u = u$, $\mathsf{U} \sqcup \mathsf{U} = \mathsf{U}$, $(\lambda g) \sqcup (\lambda h) = \lambda(g +\!\!+ h)$, $\mathsf{Pi}(a, f) \sqcup \mathsf{Pi}(b, g) = \mathsf{Pi}(a \sqcup b, f +\!\!+ g)$, and $u \sqcup v = \bot$ when u and v have different top-level constructors.

6.3 Information ordering

The ordering $u \leq v$ (written $\mathsf{LeCode}\ u\ v$ in the formalisation) is defined by structural recursion. On base codes, $\bot$ is below everything, $\mathsf{U} \leq \mathsf{U}$, and $\mathsf{Prop} \leq \mathsf{Prop}$. On compound codes:

$$\lambda g \leq \lambda h \quad \text{iff} \quad \text{for each } (u_i, v_i) \in g,\ \ v_i \leq \mathsf{Eval}(h, u_i)$$
$$\mathsf{Pi}(a, f) \leq \mathsf{Pi}(b, g) \quad \text{iff} \quad a \leq b \ \text{ and } \ \text{for each } (u_i, v_i) \in f,\ \ v_i \leq \mathsf{Eval}(g, u_i)$$

Codes with different top-level constructors are incomparable. The ordering on finite functions uses the *finite condition*: $g \leq h$ iff for each $(u, v) \in g$, $v \leq \mathsf{Eval}(h, u)$. For coherent functions, this is equivalent to pointwise ordering $\forall u.\, \mathsf{Eval}(g, u) \leq \mathsf{Eval}(h, u)$.

6.4 Coherence

A finite function $g = [(u_1, v_1), \ldots, (u_n, v_n)]$ represents a well-defined continuous function only if it is *coherent*: whenever two keys are *compatible* (have a common upper bound), their values must also be compatible. Formally, $\mathsf{Comp}(u, v)$ holds when u and v can be combined via the supremum (same constructor, recursively compatible), and $\mathsf{CoherentFun}(g)$ requires:

- each key u_i and value v_i is coherent (recursively),
- each value v_i is non-$\bot$,
- for all $i < j$, if $\mathsf{Comp}(u_i, u_j)$ then $\mathsf{Comp}(v_i, v_j)$.

On compound elements: $\mathsf{Coherent}(\mathsf{Pi}(a, f))$ requires $\mathsf{Coherent}(a)$ and $\mathsf{CoherentFun}(f)$, and similarly for $\mathsf{Sigma}(a, f)$. Base codes ($\bot$, U, Prop) are always coherent.

Coherence ensures that the supremum-based evaluation is monotone: if $u \leq v$ and g is coherent, then $\mathsf{Eval}(g, u) \leq \mathsf{Eval}(g, v)$.

6.5 Finite membership

The predicate $\mathsf{FinMem}(u, a)$ asserts that code u is a valid *approximation to an element of type* a. Its key clauses are:

$$\begin{aligned}
\mathsf{FinMem}(\bot, a) &= \mathsf{FinMem}(a, \mathsf{U}) \\
\mathsf{FinMem}(\mathsf{U}, \mathsf{U}) &= \top \qquad\qquad \mathsf{FinMem}(\mathsf{Prop}, \mathsf{U}) = \top \\
\mathsf{FinMem}(\mathsf{Pi}(a, f), \mathsf{U}) &= \mathsf{FinMem}(a, \mathsf{U}) \ \wedge\ \mathsf{FinMemAllU}(f, a) \ \wedge\ \mathsf{CoherentFun}(f) \\
\mathsf{FinMem}(\lambda g, \mathsf{Pi}(a, f)) &= \mathsf{FinMemFun}(g, a, f) \ \wedge\ \mathsf{CoherentFun}(g) \ \wedge\ \mathsf{FinMem}(\mathsf{Pi}(a, f), \mathsf{U})
\end{aligned}$$

Here $\mathsf{FinMemFun}(g, a, f)$ requires that for each $(u_i, v_i) \in g$, $\mathsf{FinMem}(u_i, a)$ and $\mathsf{FinMem}(v_i, \mathsf{Eval}(f, u_i))$; and $\mathsf{FinMemAllU}(f, a)$ requires that each pair in f has its key in a and its value in U. The clause $\mathsf{FinMem}(\bot, a) = \mathsf{FinMem}(a, \mathsf{U})$ means that $\bot$ is a valid approximation in any well-formed type.

6.6 Key properties

The following lemmas, all proved in the Agda formalisation without postulates, are central to the adequacy proof:

- **Sup closure:** $\mathsf{FinMem}(u, a)$ and $\mathsf{FinMem}(v, a)$ imply $\mathsf{FinMem}(u \sqcup v, a)$.
- **Coherent evaluation:** If $\mathsf{Coherent}(\lambda f)$ and $\mathsf{Coherent}(u)$, then $\mathsf{Coherent}(\mathsf{Eval}(f, u))$.
- **Monotonicity of FinMem:** If $u \leq v$ and $\mathsf{FinMem}(v, a)$, then $\mathsf{FinMem}(u, a)$.
- **Comp-down:** If $u \leq u'$ and $\mathsf{Comp}(u', v)$, then $\mathsf{Comp}(u, v)$.

6.7 Termination

All definitions in the mutual block (LeCode, LeFunCode, Eval, Comp, CompFun, Coherent, FinMem, and their auxiliaries) are terminating, although Agda's termination checker requires a `TERMINATING` pragma for the non-structural cases. We sketch the informal arguments.

LeCode and Comp. These are defined by pattern-matching on the pair of constructors. Same-constructor cases recurse structurally: $\mathsf{Pi}(a, f) \leq \mathsf{Pi}(b, g)$ recurses on (a, b) and on $\mathsf{LeFunCode}(f, g)$, which in turn checks $v_i \leq \mathsf{Eval}(g, u_i)$ for each $(u_i, v_i) \in f$. The result $\mathsf{Eval}(g, u_i)$ is built from values of entries of g glued by $\sqcup$; it is bounded in size by g. Meanwhile v_i is a strict subterm of the first argument. So all recursive calls operate on strictly smaller data in the total-size measure rk. Cross-constructor cases return $\bot$ (empty type) immediately. The same argument applies verbatim to Comp and CompFun.

Coherent. $\mathsf{Coherent}(u)$ is defined by structural recursion on u. On $\mathsf{Pi}(a, f)$ it recurses into $\mathsf{Coherent}(a)$ and $\mathsf{CoherentFunTail}(f)$, which walks the list f checking Coherent on each key and value (strict subterms) and CoherentWith between each entry and the tail (which calls Comp, already shown terminating). No non-structural step occurs.

FinMem. The only non-structural clause is $\mathsf{FinMem}(\bot, a) = \mathsf{FinMem}(a, \mathsf{U})$, which swaps the arguments. This can fire *at most once*: after the swap, the first argument is a (the original second argument), and every clause for a non-$\bot$ first argument either returns $\top/\bot$ immediately, or recurses on strict subterms of both arguments. In particular:

- If $a = \bot$, we get $\mathsf{FinMem}(\bot, \mathsf{U}) = \mathsf{FinMem}(\mathsf{U}, \mathsf{U}) = \top$.
- If a is non-$\bot$, the subsequent recursive calls (e.g. $\mathsf{FinMem}(\mathsf{Pi}(a', f), \mathsf{U})$ reducing to $\mathsf{FinMem}(a', \mathsf{U})$) always have a structurally smaller first argument and a non-$\bot$ second argument, so the $\bot$-swap never triggers again.

The auxiliary $\mathsf{FinMemFun}(g, a, f)$ recurses on the list g, calling FinMem on keys and values of g (strict subterms of the original first argument) with second arguments a and $\mathsf{Eval}(f, u_i)$ (derived from subterms of the original type code). After the initial $\bot$-swap, all recursion is well-founded on the total size of the arguments.

6.8 Decidability

All the relations defined on finite elements are decidable.

LeCode. Decidability is built into the definition via the numeric function $\mathsf{leFinEl} : \mathsf{Code} \to \mathsf{Code} \to \mathbb{N}$, which mirrors the structure of LeCode but returns a natural number (0 or positive). Soundness and completeness lemmas establish that $\mathsf{leFinEl}(u, v) > 0$ if and only if $\mathsf{LeCode}(u, v)$.

Comp, Coherent, FinMem. The file `FinMemDecidable.agda` provides decision procedures for the remaining predicates, using $\mathsf{Dec}\ A = A + (A \to \bot)$:

$$\mathsf{decComp} : (u\ v : \mathsf{Code}) \to \mathsf{Dec}(\mathsf{Comp}(u, v))$$
$$\mathsf{decCoherent} : (u : \mathsf{Code}) \to \mathsf{Dec}(\mathsf{Coherent}(u))$$
$$\mathsf{decFinMem} : (u\ a : \mathsf{Code}) \to \mathsf{Dec}(\mathsf{FinMem}(u, a))$$

Each decision procedure follows the same case structure as the corresponding definition, composing decisions through generic combinators for pairs (dec-Pair), disjunctions (dec-Or), and implications (dec-impl, using decidability of the premise to either produce a witness or refute by applying the function to it). All proofs are postulate-free.

7 The logical relation

The logical relation connects the domain-theoretic model to the syntax. It is defined by mutual recursion on codes, with six mutually defined predicates. We describe the main ones.

7.1 Term validity

$\mathsf{Val}\ \Gamma\ M\ A\ u\ a$ asserts that term M of type A in context Γ is valid at value code u and type code a. The definition is by case analysis on the pair (u, a):

- $\mathsf{Val}\ \Gamma\ M\ A\ u\ \bot = \top$ (trivially valid at bottom type).
- At the universe: $\mathsf{Val}\ \Gamma\ M\ A\ u\ \mathsf{U}$ is non-trivial only when $u \in \{\mathsf{U}, \mathsf{Prop}, \mathsf{Pi}(b, f), \mathsf{Sigma}(b, f)\}$, in which case it requires $\mathsf{ValTy}\ \Gamma\ A\ \mathsf{U}$ (the type A is valid at U) together with $\mathsf{ValTy}\ \Gamma\ M\ u$ (the term M is a valid type code u).
- At a Pi-type: $\mathsf{Val}\ \Gamma\ M\ A\ (\lambda g)\ (\mathsf{Pi}(b, f))$ requires $\mathsf{ValTy}\ \Gamma\ A\ (\mathsf{Pi}(b, f))$ together with application data: for each $(u_0, v_0) \in g$ and argument N with $\Gamma \vdash N : A_0$ and $\mathsf{Val}\ \Gamma\ N\ A_0\ u_0\ b$, one has $\mathsf{Val}\ \Gamma\ (M\ N)\ (B_0[N])\ v_0\ \mathsf{Eval}(f, u_0)$.
- At a Sigma-type: $\mathsf{Val}\ \Gamma\ M\ A\ (u', v')\ (\mathsf{Sigma}(b, f))$ requires $\mathsf{ValTy}\ \Gamma\ A\ (\mathsf{Sigma}(b, f))$ together with validity of both projections (see Section 11 below).
- All other combinations are trivially $\top$.

7.2 Type validity

$\mathsf{ValTy}\ \Gamma\ M\ a$ asserts that M is a valid type at code a. The interesting cases are:

- $\mathsf{ValTy}\ \Gamma\ M\ \mathsf{U}$ requires a head reduction $M \rightsquigarrow^* \mathsf{U}$ together with $\Gamma \vdash M = \mathsf{U} : \mathsf{U}$.
- $\mathsf{ValTy}\ \Gamma\ M\ (\mathsf{Pi}(b, f))$ unpacks to specific types A, B with $M \rightsquigarrow^* \Pi(x{:}A)B$, typing evidence $\Gamma \vdash A : \mathsf{U}$ and $\Gamma, x{:}A \vdash B : \mathsf{U}$, domain validity $\mathsf{ValTy}\ \Gamma\ A\ b$, and *edge functions*: for each $(u, v) \in f$ and well-typed argument N, $\mathsf{ValTy}\ \Gamma\ B[N]\ v$.
- $\mathsf{ValTy}\ \Gamma\ M\ (\mathsf{Sigma}(b, f))$ is analogous, with $M \rightsquigarrow^* \Sigma(x{:}A)B$.

7.3 Equality

$\mathsf{EqVal}\ \Gamma\ M\ N\ A\ u\ a$ bundles Val for both M and N together with an equality witness. It follows the same case structure. $\mathsf{EqValTy}\ \Gamma\ M\ N\ a$ is the corresponding type-level equality.

7.4 Adequacy

The *adequacy theorem* (Theorem 2 of Coquand–Huber 2018, extended to Σ) states that typability implies validity. In the two-substitution form:

Theorem 1 (Adequacy)**.** *If $\Gamma \vdash M : A$, then for any coherent environment ρ with a valid substitution σ into a context H, and any codes u, a such that M evaluates to u at ρ and A evaluates to a at ρ with $\mathsf{FinMem}(u, a)$, we have $\mathsf{Val}\ H\ \sigma(M)\ \sigma(A)\ u\ a$.*

The proof proceeds by mutual induction on the typing derivation, with a parallel statement for conversion ($\Gamma \vdash M = N : A$ implies EqVal) and a two-substitution variant (M under two related substitutions gives EqVal).

The injectivity results and subject reduction below are *corollaries* of adequacy: one evaluates at the trivial bottom environment $\rho_\bot$ to extract head-reduction and conversion data from the logical relation.

8 Pi injectivity

Theorem 2 (Pi injectivity — Corollary 6)**.** *If* $\Gamma \vdash A_0 = \Pi(x{:}B_1)F_1 : \mathsf{U}$*, then there exist* B_0*,* F_0 *such that*

1. $A_0 \rightsquigarrow^* \Pi(x{:}B_0)F_0$*,*
2. $\Gamma \vdash B_0 = B_1 : \mathsf{U}$*,*
3. $\Gamma, x{:}B_0 \vdash F_0 = F_1 : \mathsf{U}$*.*

Corollary 3 (Pi–Pi injectivity)**.** *If* $\Gamma \vdash \Pi(x{:}A_0)B_0 = \Pi(x{:}A_1)B_1 : \mathsf{U}$*, then*

$$\Gamma \vdash A_0 = A_1 : \mathsf{U} \qquad \textit{and} \qquad \Gamma, x{:}A_0 \vdash B_0 = B_1 : \mathsf{U}.$$

Proof outline. The proof proceeds by domain-theoretic adequacy:

1. From the conversion, extract $\Gamma \vdash A_0 : \mathsf{U}$ by presupposition.
2. Apply conversion soundness at the bottom environment $\rho_\bot$ to transfer $\mathsf{EvalRel}(\Pi B_1\, F_1, \rho_\bot, \mathsf{Pi}(\bot, \mathsf{nil}))$ to $\mathsf{EvalRel}(A_0, \rho_\bot, \mathsf{Pi}(\bot, \mathsf{nil}))$.
3. Apply bundled equality adequacy (`adequacyEqSub2`) to obtain $\mathsf{EqVal2}$ at the $\mathsf{Pi}(\bot, \mathsf{nil})$ code, which unfolds to $\mathsf{EqValTyPi2}$ containing the head reduction and domain/codomain conversions.
4. Since Π is a head normal form, reduction uniqueness (`Red-unique-Pi`) gives exact equalities.

□

9 Sigma injectivity

Theorem 4 (Sigma injectivity)**.** *If* $\Gamma \vdash A_0 = \Sigma(x{:}B_1)F_1 : \mathsf{U}$*, then there exist* B_0*,* F_0 *such that*

1. $A_0 \rightsquigarrow^* \Sigma(x{:}B_0)F_0$*,*
2. $\Gamma \vdash B_0 = B_1 : \mathsf{U}$*,*
3. $\Gamma, x{:}B_0 \vdash F_0 = F_1 : \mathsf{U}$*.*

Corollary 5 (Sigma–Sigma injectivity)**.** *If* $\Gamma \vdash \Sigma(x{:}A_0)B_0 = \Sigma(x{:}A_1)B_1 : \mathsf{U}$*, then*

$$\Gamma \vdash A_0 = A_1 : \mathsf{U} \qquad \textit{and} \qquad \Gamma, x{:}A_0 \vdash B_0 = B_1 : \mathsf{U}.$$

Proof outline. The proof follows the same strategy as Pi injectivity:

1. Extract $\Gamma \vdash A_0 : \mathsf{U}$ by presupposition.

2. Transfer $\mathsf{EvalRel}(\Sigma B_1\, F_1, \rho_\perp, \mathsf{Sigma}(\perp, \mathsf{nil}))$ to $\mathsf{EvalRel}(A_0, \rho_\perp, \mathsf{Sigma}(\perp, \mathsf{nil}))$ via conversion soundness.

3. Apply bundled equality adequacy to obtain EqValTySigma2 containing head reduction and domain/codomain conversions.

4. Since Σ is a head normal form, reduction uniqueness (`Red-unique-Sigma`) gives exact equalities.

□

10 Subject reduction

As an application of Pi and Sigma injectivity, we prove subject reduction for single-step head reduction. The proof requires two mutually defined theorems due to the Snd case.

Theorem 6 (Subject reduction and subject conversion)**.** *The following are proved by mutual induction on the typing derivation:*

1. ***Subject reduction.*** *If* $\Gamma \vdash M : A$ *and* $M \rightsquigarrow_1 N$*, then* $\Gamma \vdash N : A$*.*
2. ***Subject conversion.*** *If* $\Gamma \vdash M : A$ *and* $M \rightsquigarrow_1 N$*, then* $\Gamma \vdash M = N : A$*.*

The mutual definition is necessary because of the Snd case: when $M \rightsquigarrow_1 M'$ and we have $\Gamma \vdash \mathsf{snd}(M) : B[\mathsf{fst}(M)]$, subject reduction for $\mathsf{snd}(M) \rightsquigarrow_1 \mathsf{snd}(M')$ must produce $\Gamma \vdash \mathsf{snd}(M') : B[\mathsf{fst}(M)]$. But the direct application of Snd gives type $B[\mathsf{fst}(M')]$. To bridge the gap, we need $\Gamma \vdash \mathsf{fst}(M) = \mathsf{fst}(M') : A$, which requires subject conversion (not just subject reduction) applied to M.

Proof outline. By mutual induction on the typing derivation, case-splitting on the head reduction step.

Subject reduction cases:

- Conv and Prop-U: peel off and recurse.
- App with HR-App ($f \rightsquigarrow_1 f'$): induction gives $\Gamma \vdash f' : \Pi(x{:}A)B$; rebuild with App.
- App with HR-Beta ($(\lambda(x{:}A')M')\ a \rightsquigarrow_1 M'[a]$): the key case. Use Pi injectivity to extract the body typing $\Gamma, x{:}A_0 \vdash M' : B_0$ from the lambda derivation (walking through Conv layers), then apply substitution. The helper lemma `ty-Lam-body` performs this extraction.
- Fst with HR-Fst ($M \rightsquigarrow_1 M'$): induction gives $\Gamma \vdash M' : \Sigma(x{:}A)B$; rebuild with Fst.
- Fst with HR-Beta-Fst ($\mathsf{fst}(\mathsf{pair}(M, N)) \rightsquigarrow_1 M$): use Sigma injectivity to extract $\Gamma \vdash M : A$ from the MkPair derivation. The helper `ty-MkPair-components` performs this extraction.
- Snd with HR-Snd ($M \rightsquigarrow_1 M'$): induction on subject conversion gives $\Gamma \vdash M = M' : \Sigma(x{:}A)B$, hence $\Gamma \vdash \mathsf{fst}(M) = \mathsf{fst}(M') : A$ by Fst-Cong. Subject reduction gives $\Gamma \vdash \mathsf{snd}(M') : B[\mathsf{fst}(M')]$; type conversion via $\mathsf{fst}(M) = \mathsf{fst}(M')$ yields $\Gamma \vdash \mathsf{snd}(M') : B[\mathsf{fst}(M)]$.
- Snd with HR-Beta-Snd ($\mathsf{snd}(\mathsf{pair}(M, N)) \rightsquigarrow_1 N$): use Sigma injectivity and `ty-MkPair-components` to extract $\Gamma \vdash N : B[M]$, then convert using $\Gamma \vdash M = \mathsf{fst}(\mathsf{pair}(M, N)) : A$ (the symmetric Beta-Fst).

Subject conversion follows from subject reduction: given $\Gamma \vdash M : A$ and $M \rightsquigarrow_1 N$, subject reduction gives $\Gamma \vdash N : A$, and the corresponding conversion rule for the head reduction step (β, β-fst, β-snd, or congruence) gives $\Gamma \vdash M = N : A$.

Key helper lemmas:

- `ty-Lam-body`: If $\Gamma \vdash \lambda(x{:}A')M : \Pi(x{:}A)B$, then $\Gamma, x{:}A \vdash M : B$. Proved by walking through Conv (using Pi injectivity) and eliminating Prop-U (since U cannot head-reduce to Π).
- `ty-MkPair-components`: If $\Gamma \vdash \mathsf{pair}(M, N) : \Sigma(x{:}A)B$, then $\Gamma \vdash M : A$ and $\Gamma \vdash N : B[M]$. Proved by walking through Conv (using Sigma injectivity) and eliminating Prop-U.

□

11 Validity for Sigma types

We describe the extension of the logical relation (validity) to handle Σ-types. The key new difficulty compared to Π-types is in the head-reduction transport lemma.

11.1 Codes and projections

The set Code of codes includes:

$$u, v ::= \bot \mid \mathsf{U} \mid \mathsf{Prop} \mid \lambda g \mid \mathsf{Pi}(b, f) \mid \mathsf{Sigma}(b, f) \mid (u, v)$$

A code $\mathsf{Sigma}(b, f)$ represents a Σ-type whose domain has code b and whose codomain is the finite function f. A code (u, v) represents a pair value.

We use a uniform projection notation for both terms and codes:

$$M.1, \quad M.2 \qquad \text{(first and second projection of a term)}$$

$$w.1, \quad w.2 \qquad \text{(first and second projection of a code)}$$

On codes, the projections are defined by:

$$(u, v).1 = u, \qquad w.1 = \bot \text{ otherwise}$$

$$(u, v).2 = v, \qquad w.2 = \bot \text{ otherwise}$$

11.2 Term validity at Σ

Definition 7 (Term validity at Σ)**.** *We define* $\mathsf{Val}\ \Gamma\ M\ T\ w\ \mathsf{Sigma}(b, f)$ *to hold when:*

1. $\mathsf{ValTy}\ \Gamma\ T\ (\mathsf{Sigma}(b, f))$ *(type validity of T), which includes $T \rightsquigarrow^* \Sigma(x{:}A)B$ with $\Gamma \vdash A : \mathsf{U}$ and $\Gamma, x{:}A \vdash B : \mathsf{U}$, together with the edge functions;*
2. $\mathsf{Val}\ \Gamma\ M.1\ A\ w.1\ b$ *(validity of the first projection);*
3. $\mathsf{Val}\ \Gamma\ M.2\ B[M.1]\ w.2\ f(w.1)$ *(validity of the second projection).*

When w is not a pair (u, v), we have $w.1 = \bot$ and $w.2 = \bot$. Since Val at code $\bot$ is trivially $\top$, conditions (2) and (3) are automatically satisfied. This means there is no need for separate case analysis on whether w is a pair or not: the definition is *uniform* in w.

Compared to the Π-case, where term validity at $\mathsf{Pi}(b, f)$ is defined for $w = \lambda g$ and stores $\mathsf{Val}\ \Gamma\ (M\ N)\ (B[N])\ v\ (f(u))$, the type $B[N]$ does not mention M. For Σ, the second component's type $B[M.1]$ *does* mention M. This is the source of the difficulty in head-reduction transport.

11.3 Type validity at Σ

Definition 8 (Type validity at Σ)**.** $\mathsf{ValTy}\ \Gamma\ T\ (\mathsf{Sigma}(b, f))$ *holds when:*

- $T \rightsquigarrow^* \Sigma(x{:}A)B$ *and* $\Gamma \vdash T = \Sigma(x{:}A)B : \mathsf{U}$*;*
- $\Gamma \vdash A : \mathsf{U}$ *and* $\Gamma, x{:}A \vdash B : \mathsf{U}$*;*
- $\mathsf{ValTy}\ \Gamma\ A\ b$ *(domain type validity);*
- *For each* $(u, v) \in f$ *and* N *with* $\Gamma \vdash N : A$ *and* $\mathsf{Val}\ \Gamma\ N\ A\ u\ b$*, we have* $\mathsf{ValTy}\ \Gamma\ B[N]\ v$ *(edge function for values);*
- *Similarly for the equality edge.*

12 Type-equality transport

Lemma 9 (Transport along type equality)**.** *For all codes* u *and* a*:*

$$\mathsf{EqValTy}\ \Gamma\ C\ C'\ a \implies \mathsf{Val}\ \Gamma\ N\ C\ u\ a \implies \mathsf{Val}\ \Gamma\ N\ C'\ u\ a$$

Proof. By induction on a. The type parameter C appears in $\mathsf{Val}\ \Gamma\ N\ C\ u\ a$ only in two cases: $a = \mathsf{Pi}(b, f)$ with $u = \lambda g$, and $a = \mathsf{Sigma}(b, f)$ with $u = (u', v')$. In these cases the validity record stores a head reduction $C \rightsquigarrow^* \Pi(x{:}A)B$ or $C \rightsquigarrow^* \Sigma(x{:}A)B$, and $\mathsf{EqValTy}\ \Gamma\ C\ C'\ a$ provides the corresponding data for C', allowing the record to be rebuilt. In all other cases—including $a = \mathsf{U}$, where Val depends on N but not on C—the result is immediate. □

This lemma plays a central role in the Σ case of head-reduction transport (Lemma 10, Step 4), where it transports Val from type $B[M'.1]$ to type $B[M.1]$. For Π-types alone, this lemma is not needed in head-reduction transport.

13 Head-reduction transport

The central lemma for adequacy is:

Lemma 10 (Head-reduction produces equality)**.** *Given:*

- $M \rightsquigarrow^* M'$ *and* $\Gamma \vdash M = M' : T$*,*
- $\mathsf{Val}\ \Gamma\ M'\ T\ u\ a$ *(validity of the reduct),*

we can conclude:

$$\mathsf{EqVal}\ \Gamma\ M'\ M\ T\ u\ a$$

which bundles $\mathsf{Val}\ \Gamma\ M\ T\ u\ a$ *and* $\mathsf{Val}\ \Gamma\ M'\ T\ u\ a$ *and the equality* $\mathsf{EqVal}\ \Gamma\ M'\ M\ T\ u\ a$*.*

The return type is EqVal (not just Val). This is essential for the Σ case: the equality at the first projection is needed to transport the second projection's type.

13.1 The Π case

For $a = \mathsf{Pi}(b, f)$ and $u = \lambda g$, the validity data for M' stores, for each $(u_0, v_0) \in g$ and argument N:

$$\mathsf{Val}\ \Gamma\ (M'\ N)\ (B_0[N])\ v_0\ (f(u_0))$$

Since $M \rightsquigarrow^* M'$, we have $M\ N \rightsquigarrow^* M'\ N$. The type $B_0[N]$ does not mention M, so the induction hypothesis applies directly at code $(v_0, f(u_0))$:

$$\mathsf{EqVal}\ \Gamma\ (M'\ N)\ (M\ N)\ (B_0[N])\ v_0\ (f(u_0))$$

No type transport is needed.

13.2 The Σ case

For $a = \mathsf{Sigma}(b, f)$, we work with the uniform definition. The validity data for M' includes:

$$\mathsf{Val}\ \Gamma\ M'.1\ A\ w.1\ b \tag{1}$$

$$\mathsf{Val}\ \Gamma\ M'.2\ B[M'.1]\ w.2\ f(w.1) \tag{2}$$

We need to produce the same data for M, plus equality.

Step 1: First projection. From $M \rightsquigarrow^* M'$ we get $M.1 \rightsquigarrow^* M'.1$. From $\Gamma \vdash M = M' : \Sigma(x{:}A)B$ we derive (by the congruence rule for projections):

$$\Gamma \vdash M.1 = M'.1 : A$$

Together with (1), the induction hypothesis at code $(w.1, b)$ gives:

$$\mathsf{EqVal}\ \Gamma\ M'.1\ M.1\ A\ w.1\ b \tag{3}$$

This bundles $\mathsf{Val}\ \Gamma\ M.1\ A\ w.1\ b$.

Step 2: Type transport via the edge function. The equality (3) and the equality edge function from ValTy at $\mathsf{Sigma}(b, f)$ give:

$$\mathsf{EqValTy}\ \Gamma\ B[M'.1]\ B[M.1]\ f(w.1) \tag{4}$$

This is the key step: the *equality* edge function converts the equality of *values* ($M'.1 = M.1$) into an equality of *types* ($B[M'.1] = B[M.1]$). This step has no analogue in the Π case.

Note that the *value* edge (which only takes Val data) would not suffice here: we genuinely need the *equality* at the first projection to produce the type equality for the transport. This is why Lemma 10 must return EqVal rather than just Val.

Step 3: Second projection. From $M \rightsquigarrow^* M'$ we get $M.2 \rightsquigarrow^* M'.2$. The congruence rule for the second projection, applied to $\Gamma \vdash M' = M : \Sigma(x{:}A)B$ (note the swap, so that $M'.1$ appears in the type), gives:

$$\Gamma \vdash M'.2 = M.2 : B[M'.1]$$

Together with (2), the induction hypothesis at code $(w.2, f(w.1))$ gives:

$$\mathsf{EqVal}\ \Gamma\ M'.2\ M.2\ B[M'.1]\ w.2\ f(w.1) \tag{5}$$

In particular, $\mathsf{Val}\ \Gamma\ M.2\ B[M'.1]\ w.2\ f(w.1)$.

Step 4: Transport. The validity data for M at Σ requires:

$$\mathsf{Val}\ \Gamma\ M.2\ B[M.1]\ w.2\ f(w.1)$$

We have Val at $B[M'.1]$ from Step 3, and $\mathsf{EqValTy}$ between $B[M'.1]$ and $B[M.1]$ from Step 2. Lemma 9 gives $\mathsf{Val}\ \Gamma\ M.2\ B[M.1]\ w.2\ f(w.1)$.

14 Bundling requirements

For Π, U, and Prop alone, the following simpler choices suffice:

- Head reduction (Red) stores only $T \rightsquigarrow^* \Pi(x{:}A)B$, without the convertibility judgment $\Gamma \vdash T = \Pi(x{:}A)B : \mathsf{U}$.
- The head-reduction transport lemma can return just Val (validity of the expanded term), without the equality EqVal.
- Term validity $\mathsf{Val}\ \Gamma\ M\ T\ u\ a$ need not be bundled with type validity $\mathsf{ValTy}\ \Gamma\ T\ a$.

For Σ-types, all three must be strengthened:

1. Red **must bundle convertibility.** The head-reduction transport for Σ needs to derive $\Gamma \vdash M.1 = M'.1 : A$ (via the congruence rule for projections). This congruence rule requires $\Gamma \vdash M = M' : \Sigma(x{:}A)B$, which is obtained from the input $\Gamma \vdash M = M' : T$ by converting the type using $\Gamma \vdash T = \Sigma(x{:}A)B : \mathsf{U}$—the convertibility stored in the type validity record alongside the head reduction $T \rightsquigarrow^* \Sigma(x{:}A)B$.
2. **Head-reduction transport must return** EqVal**.** The equality EqVal at the first projection is needed to invoke the equality edge function (Step 2), which produces the type equality $\mathsf{EqValTy}\ \Gamma\ B[M'.1]\ B[M.1]\ f(w.1)$ used in the transport (Step 4).
3. Val **must be bundled with** ValTy**.** Step 2 uses the equality edge function from $\mathsf{ValTy}\ \Gamma\ T\ (\mathsf{Sigma}(b, f))$. This must be accessible from within the term validity data. Therefore $\mathsf{Val}\ \Gamma\ M\ T\ w\ (\mathsf{Sigma}(b, f))$ must include $\mathsf{ValTy}\ \Gamma\ T\ (\mathsf{Sigma}(b, f))$ as a component.

14.1 Comparison with Π

	Π	Σ
Value code	λg	any w (uniform via $w.1/w.2$)
Stored data	$\mathsf{Val}\ (M\ N)\ B[N]\ \ldots$	$\mathsf{Val}\ M.1\ A\ \ldots$ and $\mathsf{Val}\ M.2\ B[M.1]\ \ldots$
Type depends on M?	No ($B[N]$)	Yes ($B[M.1]$)
HeadRed transport	Direct IH	IH + edge + transport
Returns EqVal?	Useful	Essential
Red bundles ConvTm?	Not needed	Needed (for conv-Fst/conv-Snd)
Val bundles ValTy?	Not needed	Needed (for equality edge)

15 Agda formalization

The entire development is formalised in Agda with flags `--without-K --exact-split` (no `--type-in-type`) and **0 postulates** throughout.

Core files:

- `RawSyntax.agda` — raw terms (Π, Σ, λ, App, MkPair, Fst, Snd, U, Prop), substitution, weakening.
- `TypingRules.agda` — typing and conversion judgments.
- `PaperSemantics.agda` — trusted kernel: finite elements (FinEl, FinFun), information ordering, coherence, finite membership. No postulates.

- `RawSemantics.agda` — selection-based evaluation relation. No postulates.
- `TypingSemantics.agda` — typing and conversion soundness. 0 postulates.
- `Reduction.agda` — head reduction (HeadRed1, Red).

Validity stack (Validity5 series):

- `Validity5Core.agda` — base definitions of Val2/EqVal2 with record types.
- `Validity5DownUpRestrict.agda` — monotonicity: downward, upward, and restriction in the code argument.
- `Validity5Fwd.agda` — forward lemmas.
- `Validity5SymTrans.agda` — symmetry and transitivity for EqVal2.
- `Validity5Sup.agda` — sup closure.
- `Validity5Lemmas.agda` — collected utility lemmas.

Main results:

- `Adequacy5.agda` + `Adequacy5Cases.agda` — adequacy proof (two-substitution form). 0 postulates.
- `Adequacy5Helpers.agda`, `Adequacy5HeadRed.agda` — helper lemmas for adequacy.
- `Injectivity5.agda` — Pi and Sigma injectivity. 0 postulates.
- `SubjectReduction5.agda` — subject reduction + subject conversion (mutual). 0 postulates.

References

[1] T. Coquand and S. Huber, "An adequacy theorem for dependent type theory," *Archive for Mathematical Logic*, vol. 57, pp. 649–676, 2018.